What Does Flattening Hide? Measuring Case-Notion Distortion and Its Consequences on Seven Real Object-Centric Event Logs

Aakaash Sathya Saravanan, Mitisha Sharma, Tanmay Khambete

[Affiliation of each author: School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India. Confirm, add department and ORCID iDs.]

[Corresponding author: name and e-mail]

Abstract

Process mining wants one case per event. Plenty of real workflows give it several. One code review touches a change, a reviewer and a project, and one e-mail touches a sender, recipients and a message identifier. The usual fix is flattening: pick one object type as the case notion and copy, drop or reorder events until each one fits. Papers name the damage (convergence, divergence, deficiency), but we found no study that measures it across several real logs.

We measured it on seven public object-centric logs from code review, education, legislation, e-mail, blockchain, software commits and game play, covering 38 case notions. Then we asked what an analyst would read differently from a flat view: activity rankings, directly-follows edges, process variants and entity rankings.

The logs fall into three groups. In star logs flattening copies nothing. In partitioned logs one case notion drops 69% to 91% of the events in the median. In fan-out logs it duplicates events, up to 81.65 copies of one event. Duplication alone rarely changes a ranking: only 2 of 38 case notions fall below a rank correlation of 0.95 against the same events counted once. The bigger effect is narrowing. In 20 of 38 case notions the flat log shows fewer than half of the process's activities. And counting rows in a denormalised flat export can wreck entity rankings, with worst-case rank correlations of 0.16, -0.01 and -0.41.

Everything is recomputed from open code and reproduces byte for byte. Parts of it are checked against independent SQL, brute-force and raw-file implementations, and a plain-Python re-implementation of the distortion metrics agrees exactly on slices of three logs. Those checks caught three bugs of our own. Nobody outside the team has labelled the findings.

Keywords: object-centric process mining, OCEL 2.0, event data flattening, convergence and divergence, case notion, educational process mining.

1. Introduction

Open any process-mining tool and the first thing it asks for is a case identifier. That's fine for a purchase order walking through three desks. It's less fine for an e-mail with a sender, five recipients and a message identifier, or a Gerrit review where a change, two reviewers and a project all turn up in the same event.

The usual answer is flattening. We choose one object type as the case notion and rewrite the log so each event sits in one case. Events with several objects of that type get copied, which is convergence. Events with none get dropped, which is deficiency. Repeated events that really belong to different other objects get mixed together, which is divergence [1]. Object-centric event logs avoid the rewrite. In OCEL 2.0 [2] an event can point at any number of typed objects, and objects can point at each other.

There is good reason to think the structure matters. Keeping the object graph helps machine-learning tasks more than flattened features do [3], and Adams and van der Aalst [4] treat convergence, divergence and deficiency together. One-to-many links produced false directly-follows relations in a food-company case study [5]. Object-centric data has quality problems of its own [6]. And in education, group and resource links stay invisible in a student-centric view of Moodle activity [7].

But when we went looking for numbers, we came up short. How often does a flat view copy events? How much does it drop? Does any of that change what an analyst reads off a chart? Our searches of Zenodo, 4TU and the web turned up examples and single case studies, and no measurement across several real logs. That's the gap, and we don't claim our search was exhaustive.

This paper fills the gap in a modest way. Our contributions are three.

1. We compute distortion metrics for 38 case notions on seven real public logs and sort the logs into three regimes.
2. We compare four standard outputs (activity rankings, directly-follows edges, variants, entity rankings) between flat and object-centric views, and we keep duplication apart from narrowing, because an analyst can undo the first and can't undo the second.
3. We verify parts of the pipeline independently, report the three bugs those checks found in our own code, and make the whole study re-run with one command.

Three things we do not claim. We are not the first to use OCEL in education [7]. We say nothing

about how real capstone teams behave, because we had no portal data. And seven logs don't make a law.

2. Background and related work

The flattening problem. van der Aalst [1] explains why one-case-per-event logs misrepresent processes with interacting entities, and Adams and van der Aalst [4] address convergence, deficiency and divergence together. Both are conceptual. Neither says how often these effects occur in real data.

The format. OCEL 2.0 [2] stores events, objects, event-to-object and object-to-object relations, each with qualifiers, in JSON, XML or SQLite. We read all three forms (plus parquet bundles of the same model) and write the SQLite form.

Consequences and quality. Waibel et al. [5] used a causal event graph to show spurious directly-follows relations from one-to-many links, on one company's data. Kabierski et al. [6] classify quality issues in object-centric data. They describe the kinds of problems and don't count them. Adams et al. [3] show that structure-preserving encodings beat flattened features on predictive tasks.

Education and review processes. Miri et al. [7] ran object-centric mining on Moodle data, which makes them our closest neighbour. Hassani et al. [10] describe a department supervising more than 250 final-year computing projects, with no event data, and Premchaiswadi and Porouhan [11] mined an online peer-review process with one case notion. None of these measures flattening.

3. Method

3.1 Logs and case notions

We use seven public logs (Table 1). Five already come as OCEL files (Angular and Bundestag as JSON, Age of Empires as SQLite, Enron and Ethereum as parquet bundles). We converted OULAD [8] and CROP [9] to OCEL 2.0 ourselves. Ethereum has 3.2 million events and doesn't fit in the 8 GiB of memory we had, so we use a contiguous window of 300,000. Enron e-mail addresses are split by role (sender, recipient, copied) and hashed. A *case notion* is an object type. We call it *analysed* when its flat log covers at least 1,000 events, which leaves 38.

Table 1: The seven logs

Log	Domain	Events	Objects	Object types	Objects per event	Analysed case notions
CROP (Gerrit)	code review	306,716	55,323	3	3.0	3
OULAD	education	216,532	61,577	4	3.8	4
German Bundestag	legislation	62,719	37,665	44	1.1	9
Age of Empires II	game play	34,860	5,054	25	4.3	11
Ethereum (window)	blockchain	300,000	17,773	5	1.6	5
Angular commits	software commits	27,842	28,317	2	88.0	2
Enron e-mail	e-mail	96,175	70,541	4	3.5	4

3.2 Distortion metrics

Let E be the events, O the objects with types $\tau(o)$, and $R \subseteq E \times O$ the event-to-object relation. For a case notion T, let R_T be the pairs in R whose object has type T, and E_T the events in them. Flattening on T gives one row per pair in R_T, which is how pm4py flattens. Then:

- **Deficiency** $\delta(T) = 1 - |E_T| / |E|$. The share of events a flat view drops.
- **Convergence share** $\kappa(T)$: the share of events in E_T with two or more T-objects.
- **Copies per event** $c(T) = |R_T| / |E_T|$. It is 1.0 when nothing is copied.
- **Divergence share**: among pairs of a case and an activity with two or more events, the share whose events differ in the set of objects of at least one other type.
- **Coupling** for an ordered pair of types (T, U): the share of U-objects that take part in events of two or more T-objects.

A type U is a *nested child* of T when 95% or more of its objects occur in one T-case, each has its own lifecycle (two or more events), and the fan-out is at least 1.5 per case. These three thresholds and the 1,000-event cut-off are our choices, written at the top of the code, and we did not tune them to improve a result.

3.3 Consequence analysis

The reference is the object-centric view. Every event counts once, and the directly-follows relation is the union of consecutive-event pairs over the lifecycle of every object of every type. For each analysed case notion we compare four outputs with it.

- **C1, activity ranking.** Spearman's ρ between the flat activity counts and (a) the whole-process counts, and (b) the counts of the same covered events counted once. Measure (b) isolates duplication. Measure (a) adds narrowing. We also report the share of activities visible at all.
- **C2, directly-follows edges.** Duplication of event pairs, rank agreement of edge frequencies with and without duplicates, and, where nested children exist, the share of flat adjacent pairs whose edge type never occurs inside any child (the spurious edge

mass).

- **C3, variants.** Distinct activity sequences per case.
- **C4, entity ranking.** For every other type with at least 30 objects, ρ between the true number of events per object and the row count a denormalised flat export of *T* would give, over the objects that appear in the export.

Events are coded by sorted event identifier so ties in time always break the same way.

3.4 Verification

We checked what we could. Downloads from Zenodo are compared with the published size and MD5, and every file gets a SHA-256. A brute-force Python version of the directly-follows pairs and of one entity ranking agrees with the vectorised code on a 6,000-event slice of Enron (four case notions). Four rules of an earlier rule engine were re-implemented in SQL and match on CROP, and three counts were recomputed from the raw OULAD files. Re-running the whole study (`make study`, about five minutes on a laptop) leaves every tracked result file byte-identical. Our checks found and fixed three bugs: duplicate CROP event rows (about 17%), a missed Gerrit abandon message, and event ordering under timestamp ties. A second implementation of the distortion metrics, written with plain dictionaries and sets and no pandas, agrees exactly with the pandas code on 6,000-event slices of Enron, Bundestag and Age of Empires II (60 case notions and every type pair, divergence and variants included). What we have not done: run that second implementation on the full logs, find an independent labeller, or ask a second team to check the converters.

4. Results

4.1 Three regimes of distortion

Table 2 and Figure 1 show the distortion per log, and the logs fall into three groups.

Table 2: Flattening distortion per log, over analysed case notions

Log	Regime	Median share of events dropped	Median copies per event	Largest copies per event	Notions with over 5% duplication	Type pairs sharing at least 50% of objects
CROP (Gerrit)	star	0.0%	1.00	1.00	0 of 3	4 of 6
OULAD	star	0.0%	1.00	1.00	0 of 4	7 of 12
German Bundestag	partitioned	90.8%	1.00	1.27	2 of 9	2 of 13
Age of Empires II	partitioned	86.3%	1.01	1.76	4 of 11	50 of 137
Ethereum (window)	partitioned	68.9%	1.04	1.09	2 of 5	10 of 20
Angular commits	fan-out	0.1%	44.06	81.65	2 of 2	2 of 2
Enron e-mail	fan-out	23.1%	1.29	2.11	2 of 4	8 of 12

Star logs (CROP, OULAD). Every event touches one object of each type. Flattening copies nothing (1.00 copies per event) and, in the median, drops nothing. On these two measures the distortion is zero.

Partitioned logs (Bundestag, Age of Empires II, Ethereum). Little is copied, with at most 1.27, 1.76 and 1.09 copies per event. The catch is that different object types cover different parts of the log, so one case notion drops 69% to 91% of the events in the median.

Fan-out logs (Angular commits, Enron). One event relates to many objects of the same type. The largest duplication is 81.65 copies of an event in Angular (the branch case notion) and 2.11 in Enron.

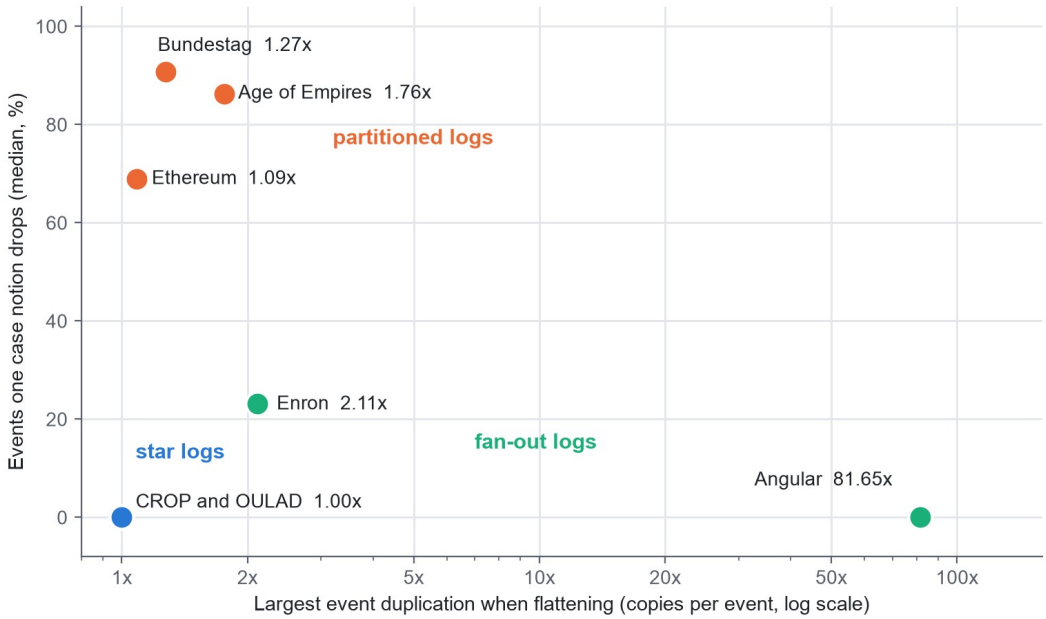


Figure 1: Duplication and dropped events for the seven logs. Each dot is one log. The horizontal axis is the largest duplication of any analysed case notion (copies per event, log scale); the vertical axis is the median share of events a case notion drops.

Coupling cuts across the groups. Apart from the Bundestag (2 of 13 type pairs) and Age of Empires II (50 of 137), at least half the type pairs have at least half of one type's objects taking part in events of two or more objects of the other, and in Angular both pairs do. The cases of these logs overlap heavily. A flat view treats them as separate.

4.2 Consequences for standard outputs

Table 3: Consequences of flattening per log

Log	Notions	Activities visible (median)	Activity rank correlation vs whole process (median / lowest)	Lowest, duplication only	Largest edge duplication	Lowest entity-rank correlation
CROP (Gerrit)	3	1.00	1.00 / 1.00	1.00	1.00	1.00
OULAD	4	1.00	1.00 / 0.87	1.00	1.00	0.98
German Bundestag	9	0.10	0.17 / 0.00	1.00	1.26	0.34
Age of Empires II	11	0.12	0.21 / -0.05	1.00	1.69	-0.41
Ethereum (window)	5	0.79	0.64 / 0.02	1.00	1.02	-0.01
Angular commits	2	0.98	0.88 / 0.81	0.81	74.05	0.88
Enron e-mail	4	1.00	0.97 / 0.87	0.93	1.57	0.16

Duplication alone rarely changes a ranking. Comparing the flat activity ranking with the ranking of the same events counted once, only 2 of the 38 case notions fall below ρ = 0.95: Angular files (0.81) and the Enron "to" role (0.93). In the star logs the correlation is exactly 1.0.

Narrowing is the bigger effect. In 20 of 38 case notions the flat log shows fewer than half of the activities of the whole process. Against the whole process, the median activity-rank correlation is 0.17 in the Bundestag and 0.21 in Age of Empires II. A flat view of those logs is not a distorted picture of the process. It is a small piece of it. Figure 2 shows both measures for every log.

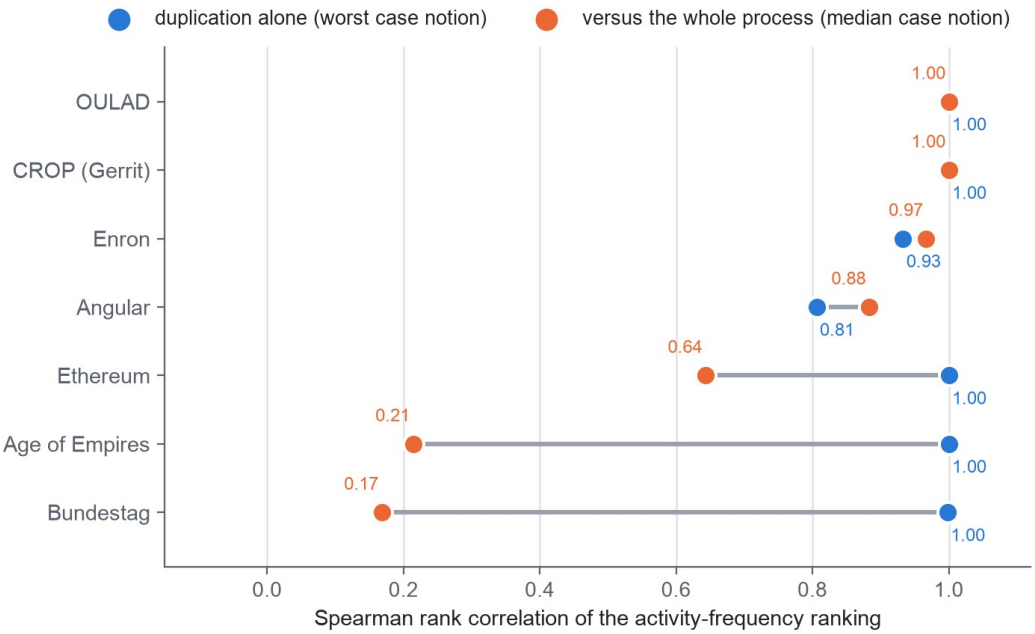


Figure 2: Activity-ranking agreement per log. Blue: duplication alone (worst case notion). Orange: against the whole process (median case notion).

Careless counting can mislead. Rank entities (people, say) by the number of rows they appear in in a denormalised flat export, and compare with the true number of events per entity. Over the entities that appear in the export (Figure 3), the worst-case ρ is 0.16 in Enron, -0.01 in Ethereum and -0.41 in Age of Empires II. An analyst who de-duplicates on the event identifier avoids this, so the finding is a warning about counting rows, and it doesn't describe a loss nobody can avoid.

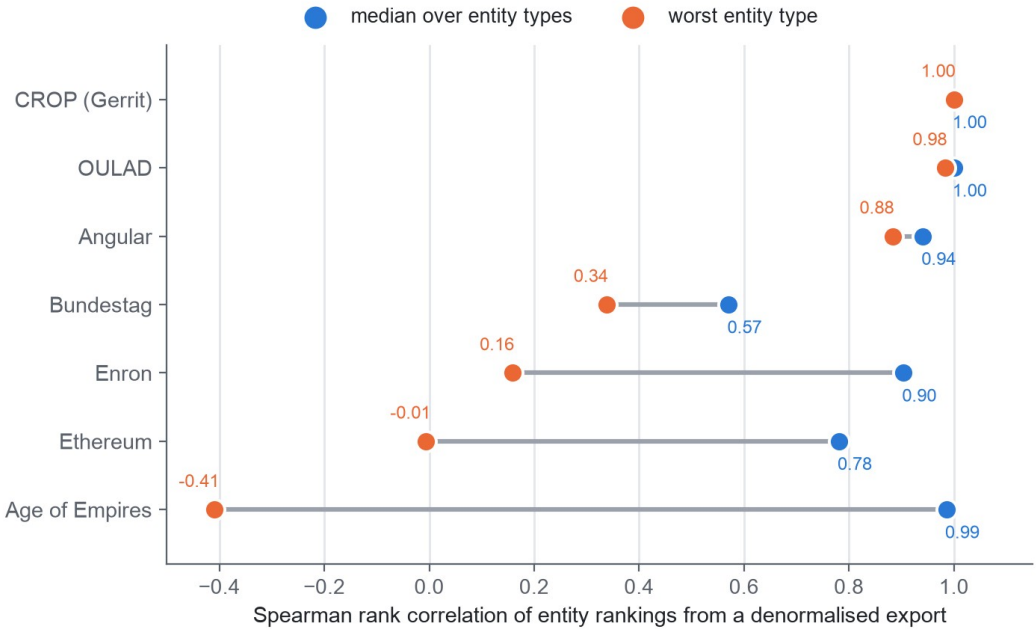


Figure 3: Entity-ranking agreement from a denormalised flat export, per log: median over entity types and worst entity type.

Edges show a smaller effect. Angular has the largest duplication of directly-follows pairs (74 copies for the branch case notion). Where nested children exist, some flat edges join events that sit next to each other in the flat log but have nothing to do with each other in the data. Those spurious edges make up at most 27.7% of adjacent pairs in Age of Empires II and 16.6% in OULAD.

5. Discussion

We started out guessing that flattening mostly duplicates events. The data disagreed. Duplication is large in only two of the seven logs. What shows up everywhere is that one case notion sees part of the process, and a flat log gives no hint of what's missing. If you take one habit from this paper, make it this: before trusting a flat result, ask which events and activities the case notion never saw.

Three practical rules follow.

9

1. Report the share of events and activities a case notion covers next to any flat result. It's two numbers, and it's the cheapest warning we found.
2. De-duplicate on the event identifier before counting anything, and count entities from the object-centric tables.
3. If objects of one type are shared across the cases of another (the coupling numbers), don't treat the cases as independent.

What this might mean for capstone review. All we have is a hypothesis. A capstone portal's documented audit log relates each action to one project and to the person who acted. If real data has that shape, flattening on the project would copy few events, but it would hide how one guide's reviews spread across many projects, and it would hide the people. That is the narrowing pattern, not the duplication pattern. We tested nothing on portal data, so we make no claim about it, and a de-identified portal log is the obvious next test.

6. Threats to validity

Seven logs. They come from several domains, but seven is seven. The three regimes are descriptive, and we don't claim they are exhaustive.

Our choices. Case notions are object types (and roles, for Enron). The nested-child thresholds and the 1,000-event cut-off are ours.

A window. The Ethereum results come from 300,000 of 3.2 million events.

A data-quality flag. About 0.1% of the Enron events (72) have timestamps before 1990, which is odd for a 1998 to 2002 corpus. We left them in.

Checking. The independent implementations are independent of the code under test, not of the team. The second implementation of the distortion metrics covers slices, not full logs. The activity-ranking numbers have no second implementation. No second person labelled the findings, and a hand review of classified CROP comments was done by one of us.

Licence. The reference flattening uses pm4py, which is AGPL-3.0. Releasing our code means choosing a compatible licence or replacing that dependency.

7. Conclusion

Flattening doesn't mainly duplicate events. In seven real logs, duplication is large only in the fan-out logs, while narrowing, where a case notion sees under half of the process's activities,

turns up in 20 of 38 case notions. Counting rows in a denormalised flat export can also reverse an entity ranking. The pipeline that produced these numbers re-runs with one command and was checked in parts by independent code, though not yet everywhere. The next steps are to run the second implementation on the full logs, independent labelling of the findings, process-discovery and prediction comparisons, a few more real logs, and, if it is ever allowed, a de-identified capstone-portal log.

Declarations

Data and code. All seven logs are public (CC BY 4.0; OULAD licence as stated by its authors [8]). Records: Zenodo 3599150 (CROP), 8430332 (Angular), 16811928 (Bundestag), 21989732 (Enron), 21989580 (Ethereum), 13365584 (Age of Empires II); UCI doi:10.24432/C5KK69 (OULAD). Code and result files are in the project repository under `realdata/`; run `make study` to regenerate every number.

Personal data. Enron e-mail addresses and CROP names are hashed on load and never printed. No VIT student, guide or portal data was used.

Synthetic data. None.

Author contributions (CRediT). [Authors to complete: conceptualisation, methodology, software, validation, writing, supervision for each author.]

Use of generative AI. [Authors to complete before submission. Most publishers require this statement. Say truthfully which AI tools were used, for what (drafting text, editing, writing code), and that the authors reviewed the output and take responsibility for the content.]

Competing interests and funding. None declared; to be confirmed by the authors.

References

[1] W. M. P. van der Aalst, "Object-centric process mining: Dealing with divergence and convergence in event data," in *Software Engineering and Formal Methods (SEFM 2019)*, Lecture Notes in Computer Science, vol. 11724. Springer, 2019, doi:10.1007/978-3-030-30446-1_1.

[2] A. Berti *et al.*, "OCEL (Object-Centric Event Log) 2.0 specification," arXiv:2403.01975, 2024.

[3] J. N. Adams, G. Park, and W. M. P. van der Aalst, "Preserving complex object-centric graph structures to improve machine learning tasks in process mining," *Engineering Applications of Artificial Intelligence*, vol. 125, art. 106764, 2023, doi:10.1016/j.engappai.2023.106764.

[4] J. N. Adams and W. M. P. van der Aalst, "Addressing convergence, divergence, and deficiency issues," in *Business Process Management Workshops (BPM 2023)*, Lecture Notes in Business Information Processing. Springer, 2024, pp. 496-507, doi:10.1007/978-3-031-50974-2_37.

[5] P. Waibel, L. Pfahlsberger, K. Revoredo, and J. Mendling, "Causal process mining from relational databases with domain knowledge," arXiv:2202.08314, 2022 (v2, 2023).

[6] M. Kabierski, M. Basmer, A. Patecka, K. Sahling, and S. Bala, "Data quality in object-centric event data: Issues classification and evaluation," *Process Science*, vol. 3, no. 1, art. 15, 2026, doi:10.1007/s44311-026-00043-x.

[7] N. Miri, S. Khayatbashi, J. Zdravkovic, and A. Jalali, "An object-centric approach to multi-entity educational process mining," *Decision Analytics Journal*, vol. 19, art. 100724, 2026, doi:10.1016/j.dajour.2026.100724.

[8] J. Kuzilek, M. Hlosta, and Z. Zdrahal, "Open University Learning Analytics dataset," *Scientific Data*, vol. 4, art. 170171, 2017, doi:10.1038/sdata.2017.171.

[9] M. Paixao, J. Krinke, D. Han, and M. Harman, "CROP: Linking code reviews to source code changes," in *Proc. 15th Int. Conf. Mining Software Repositories (MSR)*, 2018, pp. 46-49, doi:10.1145/3196398.3196466.

[10] H. Hassani *et al.*, "Supervision of undergraduate final year projects in computing: A case study," *Education Sciences*, vol. 8, no. 4, art. 210, 2018, doi:10.3390/educsci8040210.

[11] W. Premchaiswadi and P. Porouhan, "Process modeling and bottleneck mining in online peer-review systems," *SpringerPlus*, vol. 4, art. 441, 2015, doi:10.1186/s40064-015-1183-4.